
CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
CURSO SUPERIOR DE TECNOLOGIA
EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ALECSANDER MULLER GOMES

RU: 4731894

POLO DE APOIO PRESENCIAL: CURITIBA – CIC

PROFESSOR: ME. LUCIANE YANASE KANASHIRO

**DESENVOLVIMENTO DE UMA API REST PARA GERENCIAMENTO DE
PEDIDOS DO SISTEMA RAÍZES DO NORDESTE**

**Projeto Multidisciplinar apresentado ao Curso Superior de Tecnologia em
Análise e Desenvolvimento de Sistemas do Centro Universitário Internacional
UNINTER como requisito para avaliação da disciplina de Projeto
Multidisciplinar.**

Curitiba
2026

SUMÁRIO

1	INTRODUÇÃO.....	10
2	ANÁLISE E REQUISITOS.....	12
3	MODELAGEM E ARQUITETURA.....	16
4	API E ENDPOINTS.....	25
5	ENTREGA TÉCNICA.....	36
6	PLANO DE TESTES E EVIDÊNCIAS.....	42
7	CONCLUSÃO.....	49
	REFERÊNCIAS.....	51

1 INTRODUÇÃO

A utilização de sistemas informatizados no setor de alimentação tem crescido de forma significativa nos últimos anos. Restaurantes, lanchonetes e outros estabelecimentos passaram a utilizar soluções tecnológicas para automatizar processos, reduzir falhas operacionais e oferecer um atendimento mais ágil aos clientes. Nesse cenário, sistemas de gerenciamento de pedidos desempenham um papel importante, pois permitem integrar diferentes etapas do processo, desde o cadastro de usuários até o controle de estoque e o acompanhamento dos pedidos.

O presente trabalho apresenta o desenvolvimento da API REST do sistema **Raízes do Nordeste**, proposta elaborada a partir do estudo de caso apresentado na disciplina de Projeto Multidisciplinar. O sistema foi desenvolvido com o objetivo de centralizar o gerenciamento dos pedidos de uma rede de restaurantes, permitindo que diferentes canais de atendimento, como aplicativo, website, totem de autoatendimento e balcão, utilizem uma mesma estrutura de serviços.

A solução foi desenvolvida utilizando Java e o framework Spring Boot, adotando uma arquitetura em camadas para organizar as responsabilidades da aplicação. Para a persistência dos dados foi utilizado o PostgreSQL em conjunto com o Spring Data JPA, enquanto o controle de acesso foi implementado por meio de autenticação baseada em JSON Web Token (JWT), garantindo que apenas usuários autorizados possam acessar determinadas funcionalidades do sistema.

Entre os recursos desenvolvidos destacam-se o cadastro e autenticação de usuários, consulta de cardápios por unidade, gerenciamento de produtos e estoques, criação e acompanhamento de pedidos, processamento de pagamentos por meio de um serviço externo simulado (mock), programa de fidelidade com acúmulo e resgate de pontos e registro de auditoria das principais operações realizadas na aplicação.

O sistema possui dois perfis principais de usuários. O primeiro é o cliente, responsável por realizar seu cadastro, efetuar login, consultar o cardápio, realizar pedidos, acompanhar seu andamento, cancelar pedidos quando permitido e utilizar o programa de fidelidade. O segundo é o administrador, que possui acesso às

funcionalidades de gerenciamento de produtos, unidades, estoques, cardápios e atualização do status dos pedidos, além do acompanhamento das operações administrativas do sistema.

O objetivo geral deste projeto é desenvolver uma API REST capaz de atender aos requisitos propostos no estudo de caso, aplicando os conhecimentos adquiridos ao longo do curso de Análise e Desenvolvimento de Sistemas. Além disso, busca-se utilizar boas práticas de desenvolvimento de software, organização em camadas, segurança da aplicação, persistência de dados e documentação técnica.

Como objetivos específicos, destacam-se:

- Desenvolver uma API REST utilizando Spring Boot;
- Implementar autenticação e autorização utilizando JWT;
- Modelar e persistir os dados em banco PostgreSQL;
- Implementar o gerenciamento de pedidos e o controle de estoque;
- Desenvolver um programa de fidelidade baseado em acúmulo e resgate de pontos;
- Simular o processamento de pagamentos por meio de um serviço mock;
- Documentar a solução utilizando diagramas e especificação da API.

O desenvolvimento deste projeto possibilitou a aplicação prática de conceitos estudados durante a graduação, como modelagem de banco de dados, desenvolvimento de APIs REST, segurança da informação, arquitetura de software e integração entre diferentes componentes de uma aplicação. Como resultado, foi construída uma solução funcional que atende aos requisitos definidos para o estudo de caso e que pode servir como base para futuras evoluções e novas funcionalidades.

2 ANÁLISE E REQUISITOS

A etapa de análise teve como objetivo identificar as necessidades do sistema e definir os requisitos necessários para atender ao estudo de caso proposto. A partir da análise dos processos envolvidos no gerenciamento de pedidos de uma rede de restaurantes, foram definidos os requisitos funcionais e não funcionais que orientaram o desenvolvimento da aplicação.

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer aos seus usuários.

Código	Requisito	Descrição
RF01	Cadastro de usuários	Permitir o cadastro de novos clientes na plataforma.
RF02	Autenticação	Permitir que usuários realizem login utilizando e-mail e senha, retornando um token JWT para autenticação.
RF03	Controle de perfis	Diferenciar as permissões de acesso entre usuários do tipo CLIENTE e ADMIN.
RF04	Consulta de cardápio	Permitir a consulta dos produtos disponíveis em cada unidade do restaurante.
RF05	Gerenciamento	Permitir que administradores realizem cadastro, alteração

de produtos e consulta de produtos.

RF06	Gerenciamento de unidades	Permitir o cadastro e gerenciamento das unidades da rede.
RF07	Controle de estoque	Registrar e atualizar a quantidade disponível dos produtos em cada unidade.
RF08	Criação de pedidos	Permitir que clientes realizem pedidos contendo um ou mais produtos.
RF09	Atualização do status do pedido	Permitir que administradores alterem o status do pedido durante seu processamento.
RF10	Cancelamento de pedidos	Permitir o cancelamento de pedidos dentro das regras definidas pelo sistema.
RF11	Processamento de pagamento	Simular a comunicação com um serviço externo de pagamento (Mock).
RF12	Programa de fidelidade	Registrar pontos obtidos pelos clientes após pagamentos aprovados.
RF13	Resgate de pontos	Permitir que clientes utilizem seus pontos acumulados.
RF14	Auditoria	Registrar eventos importantes, como login, pagamentos, alterações de status e cancelamentos.
RF15	Canal do	Registrar a origem do pedido (APP, WEB, TOTEM,

pedido BALCÃO ou PICKUP).

2.2 Requisitos Não Funcionais

Os requisitos não funcionais definem características relacionadas ao desempenho, segurança, tecnologia e qualidade da solução.

Código	Requisito	Descrição
RNF01	Arquitetura	A aplicação deve utilizar arquitetura em camadas (Controller, Service e Repository).
RNF02	Linguagem	O sistema deve ser desenvolvido utilizando Java 21 e Spring Boot.
RNF03	Persistência	Os dados devem ser armazenados em banco PostgreSQL utilizando Spring Data JPA.
RNF04	Segurança	O acesso aos endpoints protegidos deve ocorrer por meio de autenticação JWT.
RNF05	Controle de acesso	As funcionalidades administrativas devem estar disponíveis apenas para usuários com perfil ADMIN.
RNF06	API REST	A comunicação entre cliente e servidor deve ocorrer por meio de serviços REST utilizando JSON.
RNF07	Documentação	A API deve possuir documentação automática utilizando Swagger/OpenAPI.

O sistema deve impedir operações que violem as regras
Integridade dos de negócio, como pedidos sem estoque disponível ou
RNF08 dados resgate de pontos superior ao saldo existente.

As principais operações realizadas na aplicação devem
RNF09 Auditoria ser registradas para fins de rastreabilidade.

Manutenibilidad O código deve seguir uma organização em camadas,
RNF10 e facilitando futuras manutenções e evolução do sistema.

Ao longo do desenvolvimento, todos os requisitos descritos foram considerados na implementação da aplicação. As funcionalidades foram distribuídas em módulos independentes, permitindo maior organização do código e facilitando futuras expansões, como a implementação de campanhas promocionais e integração com serviços reais de pagamento.

3 MODELAGEM E ARQUITETURA

Este capítulo apresenta a modelagem do sistema e as principais decisões arquiteturais adotadas durante o desenvolvimento da API. Foram utilizados diagramas UML e o modelo entidade-relacionamento para representar a estrutura da aplicação e o relacionamento entre seus componentes, além da descrição da arquitetura utilizada e das principais tecnologias empregadas na persistência dos dados.

3.1 Arquitetura da solução

A aplicação foi desenvolvida utilizando uma arquitetura em camadas (Layered Architecture), padrão amplamente utilizado em aplicações baseadas no framework Spring Boot. Essa abordagem permite separar as responsabilidades do sistema, facilitando a manutenção, a reutilização de código e futuras evoluções da aplicação.

A arquitetura é composta pelas seguintes camadas:

- **Controller:** responsável por receber as requisições HTTP, validar os dados enviados pelos clientes e encaminhar as solicitações para a camada de serviços.
- **Service:** concentra toda a lógica de negócio da aplicação, incluindo autenticação, gerenciamento de pedidos, controle de estoque, programa de fidelidade, processamento de pagamentos e auditoria.
- **Repository:** realiza o acesso ao banco de dados utilizando Spring Data JPA, abstraindo as operações de persistência.
- **Banco de Dados:** responsável pelo armazenamento permanente das informações do sistema, utilizando PostgreSQL.

A Figura 1 apresenta o diagrama da arquitetura da solução.

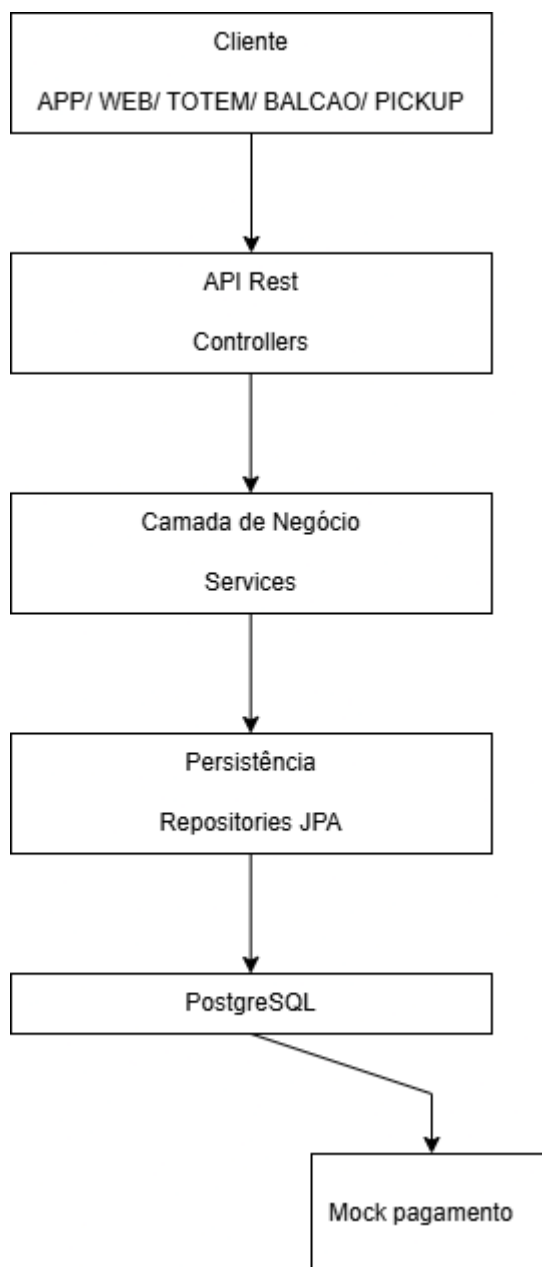


FIGURA 1: DIAGRAMA DE ARQUITETURA

3.2 Diagramas de casos de uso

O diagrama de casos de uso representa as principais funcionalidades disponibilizadas pelo sistema para seus usuários.

Foram identificados dois atores principais:

- **Cliente**, responsável por realizar cadastro, autenticação, consulta ao cardápio, criação e cancelamento de pedidos, acompanhamento do status e utilização do programa de fidelidade.
- **Administrador**, responsável pelo gerenciamento dos produtos, unidades, estoques, cardápios e atualização do status dos pedidos.

A Figura 2 apresenta o diagrama de casos de uso desenvolvido para o sistema.

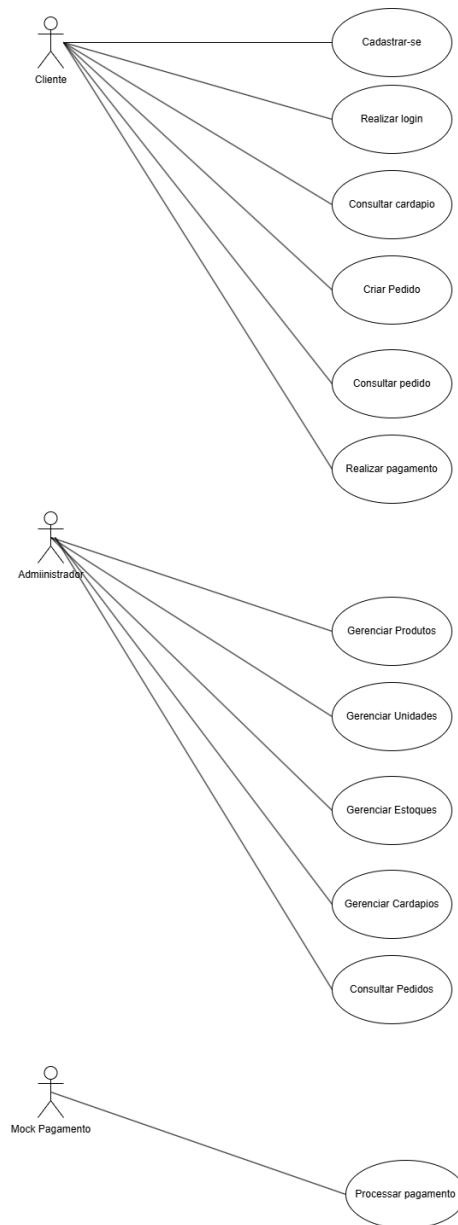


FIGURA 2: DIAGRAMA DE CASOS DE USO

3.3 Diagrama de Classes

O diagrama de classes foi elaborado para representar a estrutura estática do sistema, evidenciando as principais entidades, seus atributos e os relacionamentos existentes entre elas.

Entre as principais classes do sistema destacam-se:

- Usuario;
- Produto;
- Unidade;
- Estoque;
- Pedido;
- ItemPedido;
- Pagamento;
- FidelidadeCliente;
- MovimentacaoPontos;
- AuditLog;

O modelo também contempla os relacionamentos entre pedidos, usuários, produtos e unidades, além das entidades responsáveis pelo controle do programa de fidelidade e pelo registro das operações de auditoria.

A Figura 3 apresenta o diagrama de classes da aplicação.

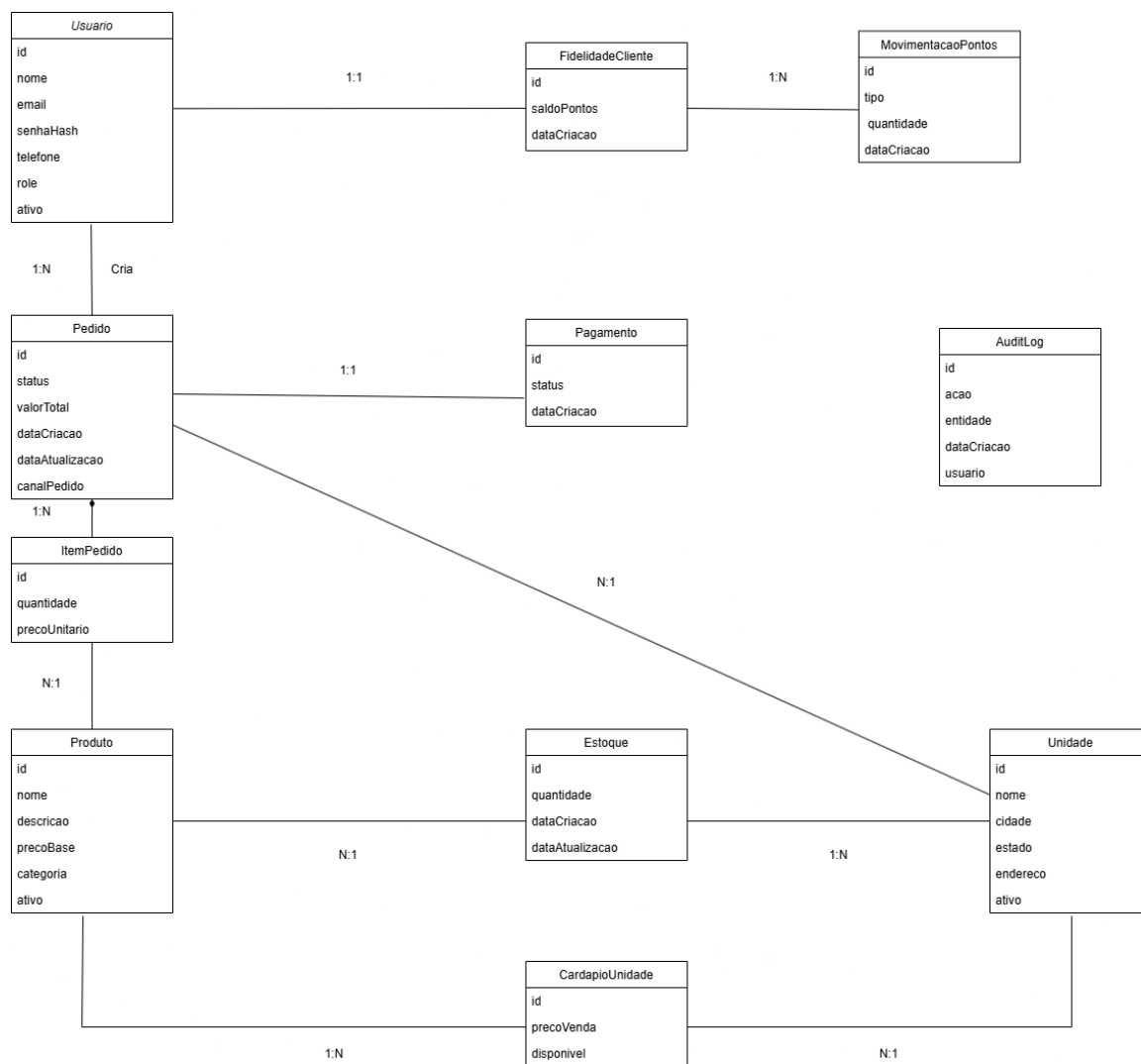


FIGURA 3: DIAGRAMA DE CLASSES

3.4 Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento foi utilizado para representar a estrutura lógica do banco de dados.

As entidades modeladas refletem diretamente as classes persistidas pela aplicação, permitindo armazenar informações relacionadas aos usuários, produtos, estoques, pedidos, pagamentos, fidelidade e auditoria.

Os relacionamentos foram definidos de forma a garantir a integridade dos dados e atender às regras de negócio estabelecidas para o sistema.

A Figura 4 apresenta o DER da aplicação.

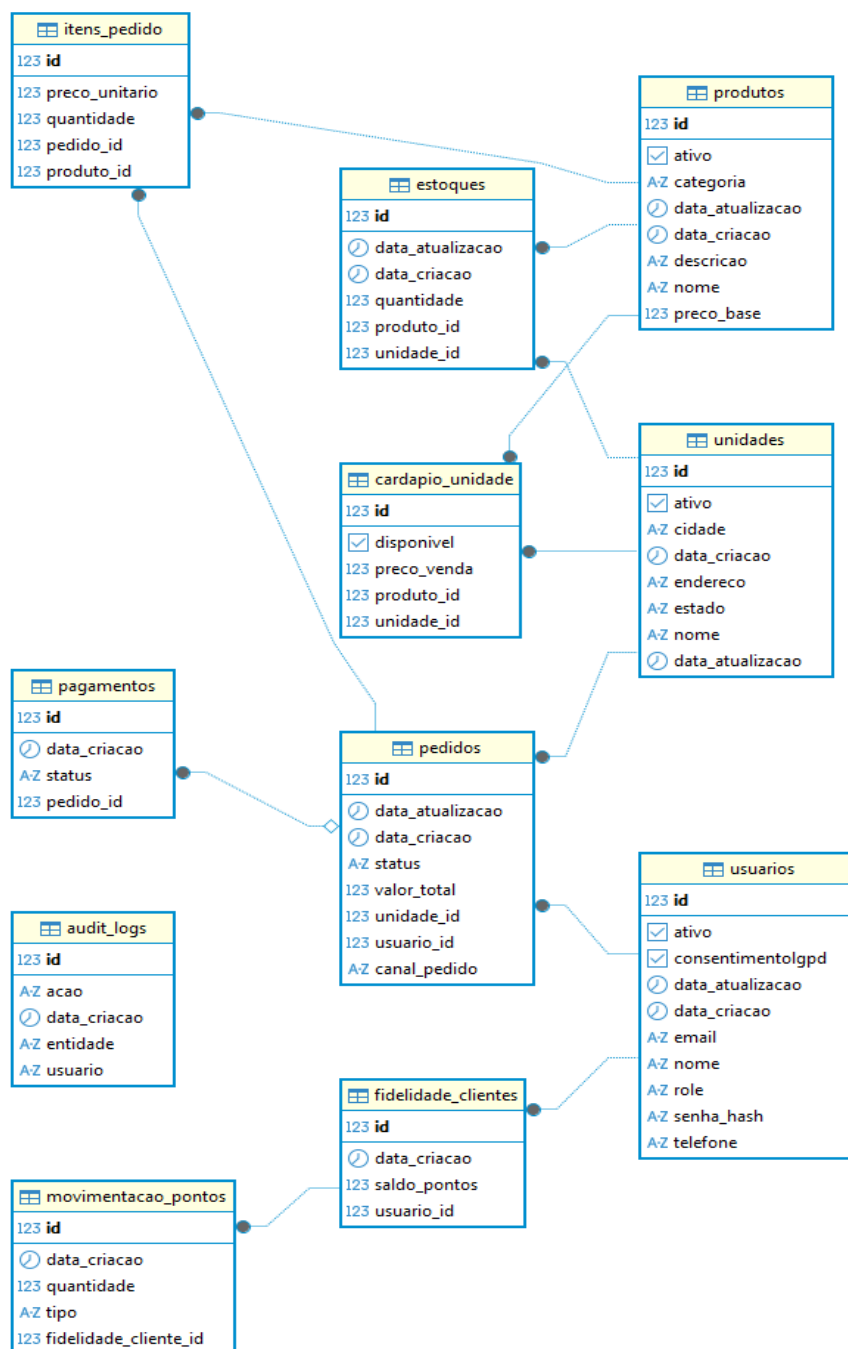


FIGURA 4: DIAGRAMA ENTIDADE-RELACIONAMENTO

3.5 Principais Endpoints da API

A API foi desenvolvida seguindo o padrão REST, utilizando requisições HTTP e troca de informações em formato JSON.

Os principais endpoints implementados são apresentados na seguinte tabela:

Método	Endpoint	Descrição
POST	/auth/register	Cadastro de usuários
POST	/auth/login	Autenticação e geração do JWT
GET	/produtos	Consulta de produtos
POST	/produtos	Cadastro de produtos
GET	/cardapios	Consulta do cardápio
POST	/pedidos	Criação de pedidos
PATCH	/pedidos/{id}/status	Atualização do status do pedido
PATCH	/pedidos/{id}/cancelar	Cancelamento do pedido
POST	/pagamentos	Processamento do pagamento
GET	/fidelidade/{usuarioid}	Consulta de pontos
POST	/fidelidade/resgatar	Resgate de pontos

A documentação completa da API foi disponibilizada por meio do Swagger/OpenAPI, permitindo a visualização dos endpoints, parâmetros e respostas retornadas pela aplicação.

3.6 Persistência de Dados

A persistência dos dados foi implementada utilizando PostgreSQL em conjunto com o Spring Data JPA.

O mapeamento objeto-relacional (ORM) foi realizado por meio do Hibernate, permitindo que as entidades Java fossem automaticamente associadas às tabelas do banco de dados.

O mapeamento objeto-relacional (ORM) foi realizado por meio do Hibernate, permitindo que as entidades Java fossem automaticamente associadas às tabelas do banco de dados.

Essa abordagem simplificou o desenvolvimento e garantiu maior produtividade durante a implementação das funcionalidades previstas no projeto.

4 API E ENDPOINTS

Este capítulo apresenta a documentação dos principais endpoints desenvolvidos para a API do sistema **Raízes do Nordeste**. O objetivo é descrever o contrato da API, especificando como cada recurso deve ser utilizado, quais informações devem ser enviadas pelo cliente e quais respostas são retornadas pela aplicação.

A API segue o padrão REST, utilizando requisições HTTP e troca de informações no formato JSON. Os recursos foram organizados por módulos, facilitando a integração com aplicações cliente, como aplicativos móveis, sistemas web e totens de autoatendimento.

4.1 Autenticação (/auth)

Cadastro de usuários

Objetivo: Permitir o cadastro de novos usuários.

Item	Descrição
Método	POST
Endpoint	/auth/register
Autenticação	Não requerida

Request

```
{  
  "nome": "João Silva",  
  "email": "joao@email.com",  
  "senha": "123456",  
  "telefone": "41999999999",
```

```
"consentimentoLGPD": true
}
```

Response (201 Created)

```
{
  "id": 1,
  "nome": "João Silva",
  "email": "joao@email.com"
}
```

Possíveis retornos

- 201 Created
- 400 Bad Request
- 409 Conflict

Login

Objetivo: Autenticar o usuário e gerar um token JWT.

Item	Descrição
Método	POST
Endpoint	/auth/login
Autenticação	Não requerida

Request

```
{
  "email": "joao@email.com",
```

```
"senha": "123456"
}
```

Response

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9..."
}
```

Possíveis retornos

- 200 OK
- 401 Unauthorized

4.2 Produtos (/produtos)

Cadastro de Produto

Objetivo: Cadastrar um novo produto.

Item	Descrição
Método	POST
Endpoint	/produtos
Autenticação	JWT
Permissão	ADMIN

Request

```
{
  "nome": "Coxinha",
```

```
"descricao": "Salgado",  
"precoBase": 8.00,  
"categoria": "Lanche"  
}
```

Response

```
{  
  "id": 1,  
  "nome": "Coxinha",  
  "precoBase": 8.00  
}
```

Status

- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden

Consulta de Produtos

Item	Descrição
Método	GET
Endpoint	/produtos
Autenticação	JWT
Permissão	ADMIN

4.3 Pedidos (/pedidos)

Criar Pedido

Objetivo: Registrar um novo pedido.

Item	Descrição
------	-----------

Método	POST
--------	------

Endpoint	/pedidos
----------	----------

Autenticação	JWT
--------------	-----

Permissão	CLIENTE
-----------	---------

Request

```
{
  "usuarioid": 1,
  "unidadeId": 1,
  "canalPedido": "APP",
  "itens": [
    {
      "produtoid": 1,
      "quantidade": 2
    }
  ]
}
```

Response

```
{  
  "id": 10,  
  "status": "RECEBIDO",  
  "valorTotal": 16.00,  
  "canalPedido": "APP"  
}
```

Validação

O campo **canalPedido** é obrigatório.

Valores permitidos:

- APP
- WEB
- TOTEM
- BALCAO
- PICKUP

Caso seja informado um valor inválido ou o campo não seja enviado, a API retorna erro de validação.

Status

- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 409 Conflict

Atualizar Status do Pedido

Item	Descrição
------	-----------

Método	PATCH
Endpoint	/pedidos/{id}/status
Autenticação	JWT
Permissão	ADMIN

Fluxo permitido:

RECEBIDO → EM_PREPARACAO → PRONTO → ENTREGUE

Cancelar Pedido

Item	Descrição
Método	PATCH
Endpoint	/pedidos/{id}/cancelar
Autenticação	JWT
Permissão	CLIENTE

O cancelamento é permitido apenas para pedidos com status RECEBIDO ou EM_PREPARACAO. Após o cancelamento, o estoque é atualizado automaticamente, devolvendo as quantidades dos produtos ao estoque da unidade correspondente.

4.4 Pagamentos (/pagamentos)

Processar Pagamento

Item	Descrição
Método	POST
Endpoint	/pagamentos
Autenticação	JWT
Permissão	CLIENTE

O pagamento é realizado por meio de um serviço externo simulado (Mock). O sistema registra o retorno da operação e atualiza o status correspondente.

Possíveis status

- APROVADO
- RECUSADO

4.5 Fidelidade (/fidelidade)

Consultar Saldo

Item	Descrição
Método	GET
Endpoint	/fidelidade/{usuarioid}
Autenticação	JWT
Permissão	CLIENTE

Response

```
{
  "usuariold": 1,
  "saldoPontos": 169
}
```

Resgatar Pontos

Item	Descrição
Método	POST
Endpoint	/fidelidade/resgatar
Autenticação	JWT
Permissão	CLIENTE

Request

```
{
  "usuariold": 1,
  "pontos": 50
}
```

Response

```
{
  "usuariold": 1,
  "saldoPontos": 119
}
```

Caso o usuário tente resgatar uma quantidade superior ao saldo disponível, a operação é recusada.

4.6 Padrão de Erros

Para manter a consistência da API, os erros seguem um formato padronizado em JSON.

Exemplo:

```
{  
  "timestamp": "2026-06-25T18:30:15",  
  "status": 400,  
  "error": "Bad Request",  
  "message": "Canal do pedido inválido.",  
  "path": "/pedidos"  
}
```

Os principais códigos de retorno utilizados pela aplicação são:

Código	Descrição
--------	-----------

200	Requisição realizada com sucesso.
-----	-----------------------------------

201	Recurso criado com sucesso.
-----	-----------------------------

400	Erro de validação dos dados enviados.
-----	---------------------------------------

401	Usuário não autenticado.
-----	--------------------------

403	Usuário autenticado, porém sem permissão para acessar o recurso.
-----	--

404	Recurso não encontrado.
-----	-------------------------

409 Violação de regra de negócio, como estoque insuficiente ou conflito de operação.

A documentação completa da API está disponível por meio do Swagger/OpenAPI, permitindo a consulta detalhada dos contratos, parâmetros e respostas de cada endpoint implementado.

5 ENTREGA TÉCNICA

Este capítulo apresenta os artefatos desenvolvidos durante o projeto, demonstrando que a aplicação atende aos requisitos técnicos definidos para a trilha de Back-end. Todos os componentes necessários para execução, validação e teste da API foram disponibilizados por meio do repositório do projeto e da documentação produzida.

5.1 Repositório do Projeto

Todo o código-fonte da aplicação foi versionado utilizando Git e disponibilizado em um repositório público, permitindo o acesso ao histórico de desenvolvimento, organização da estrutura do projeto e acompanhamento da evolução da implementação.

A organização do projeto segue a arquitetura em camadas adotada pelo Spring Boot, separando as responsabilidades entre Controllers, Services, Repositories, Entidades, DTOs, Configurações e demais componentes da aplicação. O repositório contém:

- código-fonte completo da aplicação;
- estrutura organizada por módulos e camadas;
- histórico de commits representando a evolução do desenvolvimento;
- arquivo README contendo instruções de execução do projeto;
- documentação complementar e diagramas utilizados durante a modelagem.

Link do repositório:

<https://github.com/alecsandermg/raizes-do-nordeste-api>

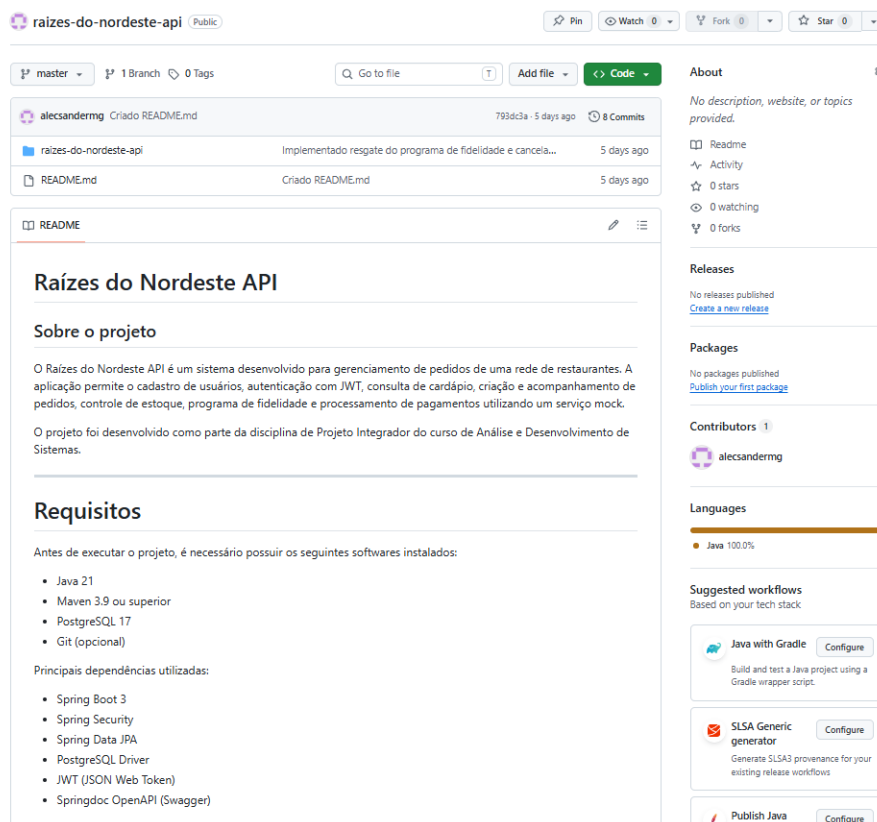


FIGURA 5: REPOSITÓRIO NO GITHUB

5.2 README de Execução

Foi elaborado um arquivo README.md contendo todas as informações necessárias para execução da aplicação em ambiente local.

O documento apresenta:

- requisitos mínimos de software;
- tecnologias utilizadas;
- configuração do banco de dados;
- configuração das variáveis de ambiente;
- instalação das dependências do projeto;
- criação da base de dados;
- execução da aplicação;
- acesso à documentação Swagger;
- orientações para realização dos testes.

Esse documento tem como objetivo permitir que qualquer desenvolvedor consiga executar a aplicação seguindo apenas as instruções disponibilizadas.

5.3 Documentação da API

A documentação da API foi disponibilizada utilizando Swagger/OpenAPI, permitindo a visualização completa dos endpoints implementados, parâmetros de entrada, modelos de dados, exemplos de requisição e resposta, além dos códigos de retorno utilizados pela aplicação.

Durante os testes também foi possível utilizar o recurso **Authorize** do Swagger para autenticação via JWT, possibilitando a execução de endpoints protegidos diretamente pela interface web.

A documentação pode ser acessada localmente através do endereço:
<http://localhost:8080/swagger-ui/index.html>

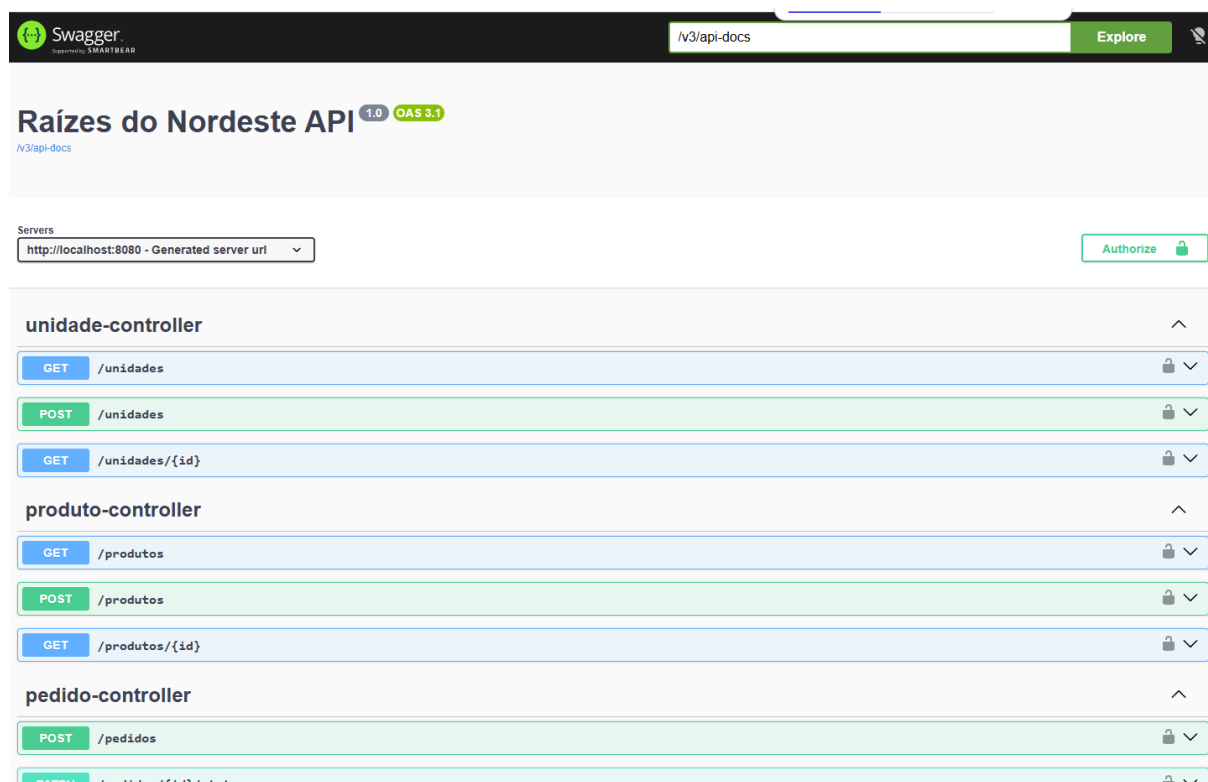


FIGURA 6: DOCUMENTAÇÃO SWAGGER

5.4 Coleção de Testes

Para validação da API foi utilizada uma coleção de requisições no Postman contendo os principais fluxos implementados.

Entre os cenários testados destacam-se:

- cadastro de usuários;
- autenticação e geração do token JWT;
- utilização do token em endpoints protegidos;
- cadastro de produtos;
- consulta de produtos e cardápios;
- criação de pedidos;
- atualização do status dos pedidos;
- cancelamento de pedidos;
- processamento de pagamentos utilizando serviço mock;
- consulta e resgate de pontos do programa de fidelidade;
- cenários de erro relacionados à autenticação, autorização, estoque insuficiente e saldo insuficiente de pontos.

A coleção utilizada nos testes encontra-se disponível em:

<https://www.postman.com/science-astronaut-66070443-s-team/workspace/my-workspace/collection/40144222-c4f810e4-5f34-48bd-a942-0b7c16a67ce5?action=share&source=copy-link&creator=40144222>

5.5 Banco de Dados

A aplicação utiliza o PostgreSQL como sistema gerenciador de banco de dados, sendo o acesso realizado por meio do Spring Data JPA e Hibernate.

A estrutura do banco foi modelada utilizando o Diagrama Entidade-Relacionamento (DER), contemplando todas as tabelas utilizadas pela aplicação.

As principais entidades persistidas são:

- usuários;
- produtos;
- unidades;
- estoques;
- cardápio por unidade;
- pedidos;
- itens do pedido;

-
- pagamentos;
 - fidelidade de clientes;
 - movimentação de pontos;
 - auditoria.

O modelo de dados apresentado no DER corresponde à estrutura efetivamente utilizada pela aplicação durante sua execução.

5.6 Fluxo Principal Implementado

Como fluxo principal da aplicação foi implementado o processo completo de gerenciamento de pedidos, atendendo ao fluxo recomendado pelo roteiro da disciplina.

O fluxo executado é composto pelas seguintes etapas:

1. autenticação do usuário utilizando JWT;
2. criação do pedido;
3. validação da disponibilidade dos produtos em estoque;
4. baixa automática do estoque da unidade selecionada;
5. processamento do pagamento utilizando um serviço mock;
6. atualização do status do pedido;
7. registro da auditoria da operação;
8. geração de pontos no programa de fidelidade após pagamento aprovado.

Além do fluxo principal, também foi implementado o cancelamento de pedidos. Quando permitido pelas regras de negócio, o sistema devolve automaticamente os itens ao estoque da unidade correspondente e registra a operação na auditoria.

Essas funcionalidades demonstram o funcionamento integrado dos principais módulos da aplicação, evidenciando a persistência dos dados, aplicação das regras de negócio, autenticação dos usuários e comunicação entre as diferentes camadas da arquitetura.

6 PLANO DE TESTES E EVIDÊNCIAS

A validação da aplicação foi realizada durante todo o processo de desenvolvimento, utilizando os recursos disponibilizados pelo Swagger/OpenAPI e pelo Postman para verificar o correto funcionamento dos endpoints implementados.

Os testes tiveram como objetivo validar as principais regras de negócio da aplicação, o funcionamento da autenticação, o controle de permissões entre perfis de usuários, a persistência dos dados no banco PostgreSQL e o comportamento da API diante de cenários de sucesso e de erro.

6.1 Estratégia de Validação

A estratégia de testes adotada consistiu na execução manual dos principais fluxos da aplicação, simulando situações que representam o uso esperado do sistema.

Os testes contemplaram:

- autenticação de usuários utilizando JWT;
- validação das permissões de acesso para os perfis CLIENTE e ADMIN;
- operações de cadastro e consulta dos principais recursos;
- criação e gerenciamento de pedidos;
- controle de estoque durante a criação e cancelamento de pedidos;
- processamento de pagamentos utilizando serviço mock;
- programa de fidelidade;
- tratamento de erros e regras de negócio.

Todos os testes foram executados com persistência em banco de dados PostgreSQL, garantindo que as alterações realizadas pela aplicação fossem efetivamente armazenadas.

6.2 Cenários de Teste

A Tabela abaixo apresenta alguns dos principais cenários utilizados durante a validação da aplicação.

Caso	Objetivo	Entrada	Resultado Esperado	Resultado Obtido
CT01	Cadastro de usuário	Dados válidos	Usuário cadastrado com sucesso	Sucesso
CT02	Login	E-mail e senha válidos	Retorno de token JWT	Sucesso
CT03	Login inválido	Senha incorreta	Retorno de erro de autenticação	Sucesso
CT04	Cadastro de produto	Usuário ADMIN autenticado	Produto cadastrado	Sucesso
CT05	Cadastro de produto	Usuário CLIENTE autenticado	Acesso negado (403)	Sucesso
CT06	Criação de pedido	Produtos com estoque disponível	Pedido criado	Sucesso
CT07	Criação de pedido	Estoque insuficiente	Operação recusada	Sucesso
CT08	Atualização de status	Usuário ADMIN	Status atualizado	Sucesso
CT09	Atualização de status	Usuário CLIENTE	Acesso negado (403)	Sucesso
CT10	Cancelamento de pedido	Pedido em status permitido	Pedido cancelado e estoque devolvido	Sucesso
CT11	Pagamento	Pedido válido	Pagamento processado	Sucesso

			pelo mock	
			Saldo de pontos retornado	
CT12	Consulta de fidelidade	Usuário autenticado		Sucesso
			Pontos resgatados	
CT13	Resgate de pontos	Saldo suficiente		Sucesso
			Operação recusada	
CT14	Resgate de pontos	Saldo insuficiente		Sucesso
			Registro gravado na tabela de auditoria	
CT15	Auditoria	Login, pagamento e cancelamento		Sucesso

6.3 Critérios de Aceitação

Os testes foram considerados aprovados quando atenderam aos seguintes critérios:

- retorno do código HTTP esperado;
- persistência correta das informações no banco de dados;
- aplicação das regras de negócio definidas para cada operação;
- validação das permissões de acesso conforme o perfil do usuário;
- retorno de mensagens de erro quando aplicável;
- atualização correta dos registros relacionados, como estoque, pagamentos, auditoria e programa de fidelidade.

6.4 Evidências de Execução

As evidências da execução dos testes foram obtidas durante o desenvolvimento da aplicação e são apresentadas por meio de capturas de tela e registros das ferramentas utilizadas.

As principais evidências incluem:

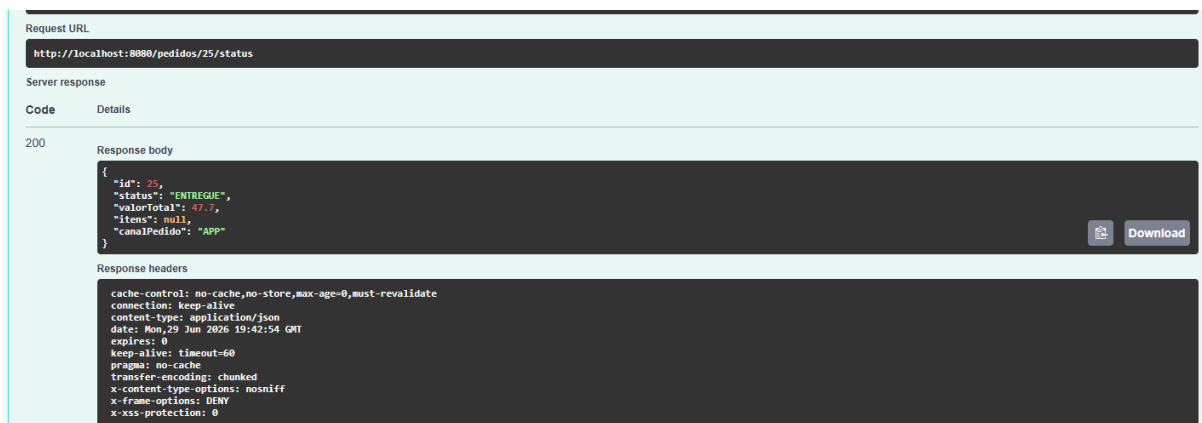


FIGURA 9: ATUALIZAÇÃO DO STATUS DO PEDIDO

- processamento de pagamentos utilizando serviço mock;

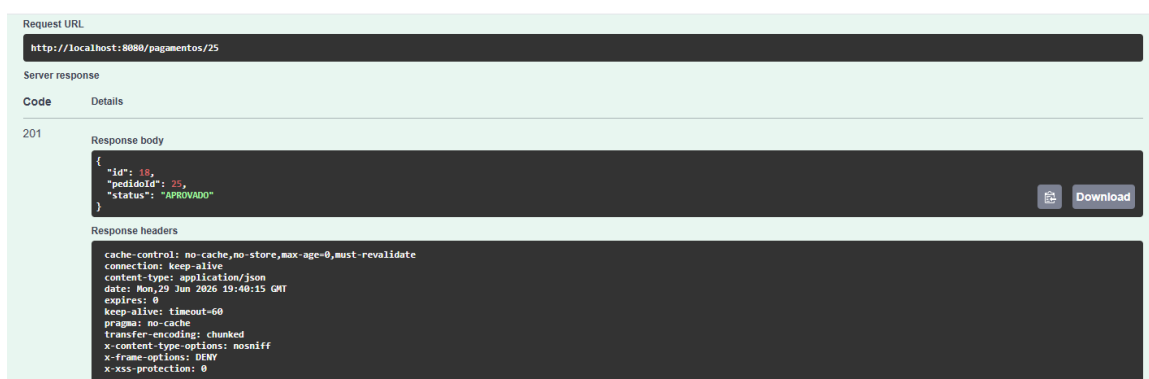


FIGURA 10: PROCESSAMENTO DE PAGAMENTO

- consulta e resgate de pontos do programa de fidelidade;

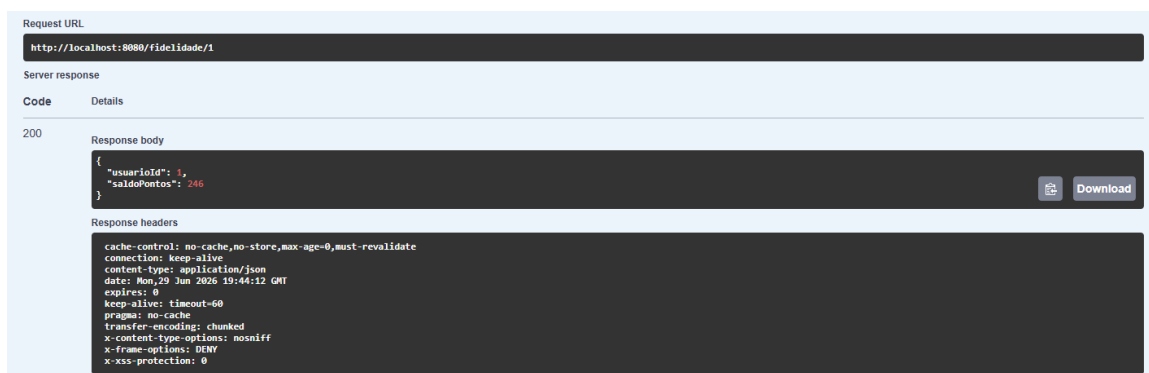


FIGURA 11: CONSULTA DE PONTOS DE FIDELIDADE

- registros de auditoria gravados no banco de dados;

```
raizes_db=# SELECT * FROM audit_logs;
```

id	acao	data_criacao	entidade	usuario
1	APROVADO	2026-06-19 15:53:02.952514	PAGAMENTO	joao@email.com
2	RECUSADO	2026-06-19 15:53:02.957154	PAGAMENTO	joao@email.com
3	APROVADO	2026-06-19 15:57:28.589143	PAGAMENTO	joao@email.com
4	RECUSADO	2026-06-19 15:57:28.592113	PAGAMENTO	joao@email.com
5	RECUSADO	2026-06-19 16:00:25.503468	PAGAMENTO	joao@email.com
6	APROVADO	2026-06-19 16:01:10.163712	PAGAMENTO	joao@email.com
7	SUCESSO	2026-06-19 16:14:45.358655	LOGIN	joao@email.com
8	FALHA	2026-06-19 16:15:13.949405	LOGIN	joao@email.com
9	SUCESSO	2026-06-19 16:42:40.274615	LOGIN	joao@email.com
10	SUCESSO	2026-06-22 22:21:56.580882	LOGIN	joao@email.com
11	SUCESSO	2026-06-22 22:26:29.524874	LOGIN	admin@email.com
12	SUCESSO	2026-06-22 22:28:51.164213	LOGIN	admin@email.com
13	SUCESSO	2026-06-22 22:33:58.052077	LOGIN	admin@email.com
14	SUCESSO	2026-06-22 22:37:32.810345	LOGIN	admin@email.com
15	SUCESSO	2026-06-22 22:47:01.456621	LOGIN	admin@email.com
16	SUCESSO	2026-06-22 22:53:05.550896	LOGIN	admin@email.com
17	SUCESSO	2026-06-22 22:59:37.192306	LOGIN	admin@email.com
18	SUCESSO	2026-06-23 19:39:05.668789	LOGIN	joao@email.com
19	STATUS_EM_PREPARACAO	2026-06-23 19:40:13.034232	PEDIDO	joao@email.com
20	STATUS_PRONTO	2026-06-23 19:42:08.206898	PEDIDO	joao@email.com
21	STATUS_ENTREGUE	2026-06-23 19:42:16.774328	PEDIDO	joao@email.com
22	SUCESSO	2026-06-23 22:34:34.769974	LOGIN	joao@email.com
23	SUCESSO	2026-06-23 22:35:38.82565	LOGIN	admin@email.com
24	STATUS_EM_PREPARACAO	2026-06-23 22:35:58.001738	PEDIDO	joao@email.com
25	SUCESSO	2026-06-23 22:47:34.016143	LOGIN	admin@email.com
26	CANCELADO	2026-06-23 22:47:57.362141	PEDIDO	joao@email.com
27	CANCELADO	2026-06-23 22:49:14.011405	PEDIDO	joao@email.com

FIGURA 12: CONSULTA DE LOGS PARA AUDITORIA

- consultas realizadas diretamente no PostgreSQL para validação da persistência dos dados.

As Figuras apresentadas ao longo deste capítulo demonstram a execução dos principais fluxos implementados e comprovam o funcionamento da aplicação conforme os requisitos definidos para o projeto.

6.5 Considerações sobre os Testes

Os testes executados demonstraram que a aplicação atende aos requisitos funcionais definidos durante a etapa de análise. Os principais fluxos da API foram validados com sucesso, incluindo autenticação, controle de acesso, gerenciamento de pedidos, controle de estoque, processamento de pagamentos, programa de fidelidade e auditoria.

Além dos cenários de sucesso, também foram realizados testes envolvendo situações de erro, como autenticação inválida, ausência de permissões, estoque insuficiente e saldo insuficiente de pontos, verificando que a aplicação responde de forma consistente e conforme as regras de negócio estabelecidas.

7 CONCLUSÃO

O desenvolvimento da API do sistema **Raízes do Nordeste** possibilitou a aplicação prática de diversos conceitos relacionados ao desenvolvimento Back-end, como arquitetura em camadas, APIs REST, autenticação com JWT, persistência de dados utilizando PostgreSQL, modelagem de banco de dados e implementação de regras de negócio.

Ao longo do projeto foi possível desenvolver uma solução funcional, capaz de atender aos principais requisitos definidos no estudo de caso. Entre as funcionalidades implementadas destacam-se o gerenciamento de usuários, produtos, unidades, estoque, pedidos, pagamentos, programa de fidelidade e auditoria, permitindo que o sistema represente um cenário próximo ao encontrado em aplicações reais.

Além dos aspectos técnicos, o projeto proporcionou uma experiência significativa no planejamento e organização do desenvolvimento de software. A necessidade de integrar diferentes módulos e garantir o correto funcionamento entre eles exigiu atenção constante às regras de negócio, à estrutura do banco de dados e à organização do código, contribuindo para o desenvolvimento de uma visão mais ampla sobre a construção de aplicações corporativas.

Durante a execução do projeto também foram encontrados desafios importantes. Embora a disciplina tenha apresentado os conceitos fundamentais necessários para compreender o desenvolvimento Back-end, a implementação completa da solução exigiu um nível de aprofundamento superior ao abordado nas aulas. Dessa forma, foi necessário realizar pesquisas complementares, consultar documentações oficiais, estudar bibliotecas e frameworks utilizados no projeto e buscar referências adicionais para compreender tecnologias e práticas que não haviam sido exploradas com maior profundidade durante o curso.

Outro desafio observado foi o tempo disponível para o desenvolvimento do projeto. Considerando a quantidade de requisitos propostos, a elaboração da documentação, a modelagem do sistema, a implementação das funcionalidades e a realização dos testes, o período destinado à execução mostrou-se bastante

reduzido. Isso exigiu um planejamento cuidadoso e uma dedicação significativa para que todas as etapas fossem concluídas dentro do prazo estabelecido.

Apesar dessas dificuldades, o projeto contribuiu de forma expressiva para o aprendizado, permitindo consolidar conhecimentos adquiridos ao longo da graduação e desenvolver competências relacionadas à resolução de problemas, pesquisa técnica e desenvolvimento de software. A experiência demonstrou a importância da aprendizagem contínua na área de tecnologia, uma vez que o desenvolvimento profissional depende não apenas dos conteúdos apresentados em sala de aula, mas também da capacidade de estudar novas ferramentas, consultar documentação técnica e buscar soluções de forma autônoma.

Como possibilidade de evolução futura, o sistema poderá receber novas funcionalidades, como a implementação completa do módulo de promoções e campanhas, integração com serviços reais de pagamento, envio de notificações aos clientes, geração de relatórios gerenciais, monitoramento da aplicação e desenvolvimento de uma interface web ou aplicativo móvel para consumo da API.

De forma geral, os objetivos propostos para o projeto foram alcançados, resultando em uma API funcional, documentada e aderente aos requisitos definidos para o estudo de caso, além de proporcionar uma experiência prática relevante para a formação acadêmica e profissional.

REFERÊNCIAS

FOWLER, Martin. *UML Essencial: um breve guia para a linguagem padrão de modelagem de objetos*. 3. ed. Porto Alegre: Bookman, 2005.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2000.

ORACLE. *Java Platform, Standard Edition Documentation*. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 23 jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 19 jun. 2026.

SPRING. *Spring Boot Reference Documentation*. Disponível em: <https://docs.spring.io/spring-boot/documentation.html>. Acesso em: 19 jun. 2026.

SPRING. *Spring Security Reference*. Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 21 jun. 2026.

OPENAPI INITIATIVE. *OpenAPI Specification*. Disponível em: <https://swagger.io/specification/>. Acesso em: 29 jun. 2026.

SPRINGDOC. *Springdoc OpenAPI Documentation*. Disponível em: <https://springdoc.org/>. Acesso em: 20 jun. 2026.

JWT.IO. *JSON Web Tokens*. Disponível em: <https://jwt.io/>. Acesso em: 29 jun. 2026.

MAVEN APACHE PROJECT. *Apache Maven Project*. Disponível em: <https://maven.apache.org/>. Acesso em: 15 jun. 2026.

ECLIPSE FOUNDATION. *Eclipse IDE*. Disponível em: <https://www.eclipse.org/ide/>. Acesso em: 15 jun. 2026.

UNINTER. *Projeto Multidisciplinar – Roteiro de Atividade Prática: Trilha Back-end*. Curitiba: Centro Universitário Internacional UNINTER, 2026.